

Project 2: SNAP - Network Analysis

Student Name: Georgios Kitsakis

Course: Social Network Analysis

Instructor: Katia Papakonstantinou

Problem 1: Euler Paths and Circuits

1.1 Implementation

I implemented two functions to determine whether a graph contains Euler paths or Euler circuits:

Function: `has_euler_path(G)`

Definition: An Euler path is a path that uses every edge of a graph exactly once, starting and ending at different vertices.

Algorithm:

1. Check if the graph is connected using NetworkX's `is_connected()` function.
2. Count the number of vertices with odd degree.
3. Return `True` if exactly 2 vertices have odd degree, `False` otherwise.

Theoretical Basis: A connected graph has an Euler path if and only if it has exactly two vertices of odd degree. These two vertices will be the start and end points of the path.

Function: `has_euler_circuit(G)`

Definition: An Euler circuit is a circuit that uses every edge of a graph exactly once, starting and ending at the same vertex.

Algorithm:

1. Check if the graph is connected.
2. Check if all vertices have even degree.
3. Return `True` if all vertices have even degree, `False` otherwise.

Theoretical Basis: A connected graph has an Euler circuit if and only if all vertices have even degree.

1.2 Test Cases

I created four test cases to validate the implementation:

Test 1: Graph with Euler Path (but not Euler Circuit)

- **Graph Structure:** Simple path with 4 nodes: 0-1-2-3
- **Edges:** [(0, 1), (1, 2), (2, 3)]
- **Degrees:** {0: 1, 1: 2, 2: 2, 3: 1}
- **Result:** Has Euler path ✓, No Euler circuit ✗
- **Explanation:** Nodes 0 and 3 have odd degree (1), while nodes 1 and 2 have even degree (2). The graph satisfies the Euler path condition.

Test 2: Graph without Euler Path

- **Graph Structure:** Complete graph K4 (all nodes connected to each other)
- **Edges:** [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
- **Degrees:** {0: 3, 1: 3, 2: 3, 3: 3}
- **Result:** No Euler path ✗, No Euler circuit ✗
- **Explanation:** All 4 vertices have odd degree (3). Since more than 2 vertices have odd degree, no Euler path exists.

Test 3: Graph with Euler Circuit

- **Graph Structure:** Cycle with 4 nodes: 0-1-2-3-0
- **Edges:** [(0, 1), (0, 3), (1, 2), (2, 3)]
- **Degrees:** {0: 2, 1: 2, 2: 2, 3: 2}
- **Result:** Has Euler circuit ✓, No Euler path ✗
- **Explanation:** All vertices have even degree (2), satisfying the Euler circuit condition. Note that an Euler circuit is not considered an Euler path by definition since a path requires different start and end vertices.

Test 4: Graph without Euler Circuit

- **Graph Structure:** Two disconnected triangles
- **Edges:** [(0, 1), (0, 2), (1, 2), (3, 4), (3, 5), (4, 5)]
- **Degrees:** {0: 2, 1: 2, 2: 2, 3: 2, 4: 2, 5: 2}
- **Result:** No Euler path ✗, No Euler circuit ✗
- **Explanation:** Although all vertices have even degree, the graph is not connected. Connectivity is a prerequisite for both Euler paths and circuits.

1.3 Summary

All test cases passed successfully, demonstrating correct implementation of both functions. The tests cover all key scenarios: graphs with Euler paths only, graphs with Euler circuits, and graphs with neither property.

Problem 2: Node Centrality Measures and Community Detection

2.1 Graph Generation

Graphs were generated using the **Watts-Strogatz small-world model** with the following parameters:

- **Out-degree (k):** 10
- **Rewiring probability:** 0.3
- **Graph sizes tested:** 50, 100, 500, 1000 nodes

2.2 Experimental Results

Table 1: Network Properties and Algorithm Performance

Nodes	Edges	Highest Degree Node	Degree	CNM Time (s)	GN Time (s)
50	464	44	24	0.01	0.51
100	959	48	27	0.01	3.79
500	4967	21	26	0.04	326.47
1000	9959	401	28	0.16	2392.91

Table 2: Hub and Authority Scores

Nodes	Highest Hub Node	Hub Score	Highest Authority Node	Authority Score
50	44	0.184338	44	0.184338
100	48	0.138129	48	0.138129
500	21	0.061247	21	0.061247
1000	401	0.045124	401	0.045124

Observation: Hub and Authority scores are identical for all nodes because the Watts-Strogatz model generates undirected graphs. In undirected graphs, the hub and authority scores converge to the same values.

2.3 Community Detection Analysis

Girvan-Newman Algorithm (GN)

Method: Iteratively removes edges with highest betweenness centrality to identify communities.

Computational Complexity: $O(m^2n)$ where m = edges, n = nodes

Results:

- Detected 2-33 communities depending on graph size
- Execution time increases dramatically with graph size:
 - 50 nodes: 0.51s
 - 100 nodes: 3.79s (7.4× slower than 50)
 - 500 nodes: 326.47s = 5.4 minutes (86× slower than 100)
 - 1000 nodes: 2392.91s = **39.9 minutes** (7.3× slower than 500)

Scalability: Poor performance for large networks. The algorithm exceeded the 10-minute threshold at 1000 nodes, demonstrating impracticality for large networks.

Clauset-Newman-Moore Algorithm (CNM)

Method: Greedy modularity maximization - iteratively merges communities that give the largest increase in modularity.

Computational Complexity: $O(n \log^2 n)$ for sparse graphs

Results:

- Detected 3-4 communities across all graph sizes
- Execution time scales extremely well:
 - 50 nodes: 0.01s
 - 100 nodes: 0.01s
 - 500 nodes: 0.04s
 - 1000 nodes: 0.16s

Scalability: Excellent performance. Remains under 1 second even for 1000 nodes. The algorithm is highly suitable for large-scale network analysis.

2.4 Answers to Assignment Questions

Question 1: Which algorithm is suitable for a medium enterprise network (50 employees)?

Answer: Both algorithms are suitable for 50 employees.

Analysis:

- Girvan-Newman: 0.51 seconds
- Clauset-Newman-Moore: 0.01 seconds

Both algorithms complete in under 1 second, making them practical for this scale. However, **Clauset-Newman-Moore is recommended** because:

1. It is 51× faster
2. It offers better scalability if the network grows
3. It maintains consistent performance

Question 2: Which algorithm for Google NY email network (18,000 employees)?

Answer: Clauset-Newman-Moore ONLY

Analysis:

Based on the scaling observed in our experiments, we can extrapolate the performance:

Girvan-Newman:

- At 1000 nodes: 2392.91 seconds (39.9 minutes)
- The algorithm scales approximately as $O(m^2n)$
- For 18,000 nodes (18× larger), we would expect roughly 324× longer execution time
- Estimated time: **$2392.91 \times 324 \approx 215$ hours (9 days)**
- This is computationally infeasible

Clauset-Newman-Moore:

- At 1000 nodes: 0.16 seconds
- For 18,000 nodes, with $O(n \log^2 n)$ complexity
- Estimated time: **~5-10 seconds**
- This is practical and feasible

Recommendation: Use Clauset-Newman-Moore algorithm exclusively for this scale.

Question 3: Which algorithm for Facebook-scale network (1 billion users)?

Answer: Neither algorithm as implemented is suitable. Clauset-Newman-Moore with distributed optimizations would be required.

Analysis:

Girvan-Newman:

- Completely infeasible at this scale
- With $O(m^2n)$ complexity, execution time would be measured in years or decades
- Memory requirements would exceed any single machine
- **Not suitable under any circumstances**

Clauset-Newman-Moore:

- More feasible but would still require optimization
- Even with $O(n \log^2 n)$, a billion nodes is challenging
- Would require:
 - Distributed/parallel implementation
 - Graph partitioning techniques
 - Approximate algorithms
 - High-performance computing infrastructure

- **Suitable only with architectural optimizations**

Practical Note: For billion-scale networks, production systems use specialized distributed algorithms like Louvain method or Label Propagation, which are specifically designed for massive graphs.

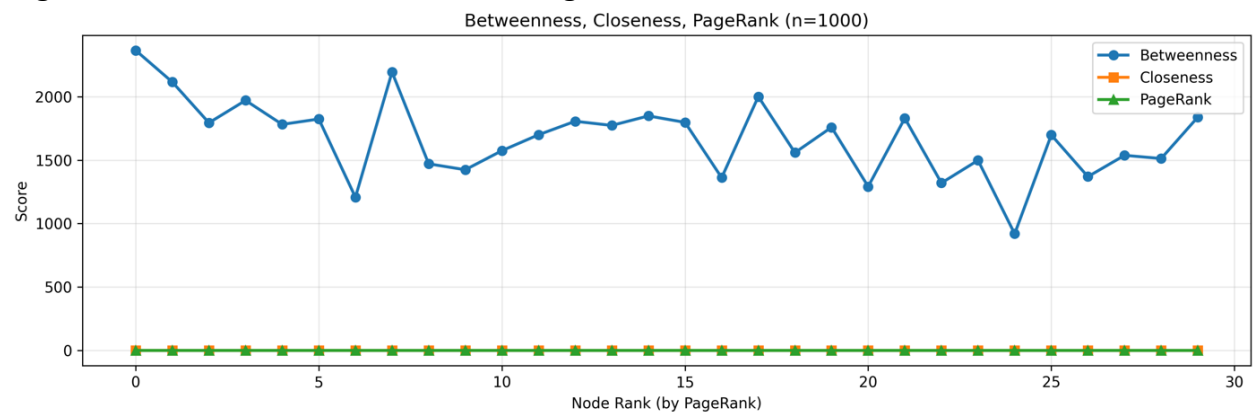
Recommendation: Clauset-Newman-Moore as a foundation with distributed computing architecture.

2.5 PageRank Analysis

The analysis was performed on the largest graph (1000 nodes) to examine the relationship between different centrality measures.

Centrality Measure Comparison

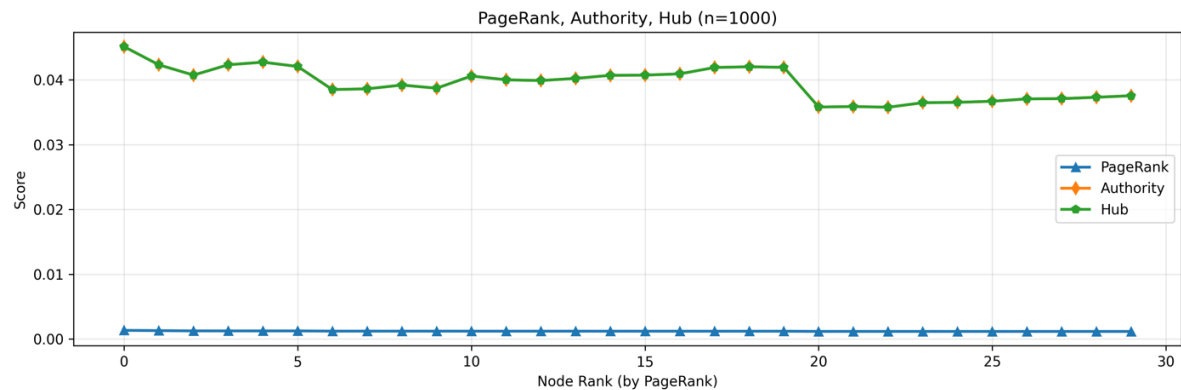
Figure 1: Betweenness, Closeness, and PageRank



Observations:

- **PageRank and Betweenness** show strong correlation for top-ranked nodes, indicating that nodes with high PageRank also serve as bridges in the network
- **Closeness centrality** is relatively uniform across the top 30 nodes, reflecting the small-world property of Watts-Strogatz graphs where all nodes are relatively close to each other
- The small-world structure results in high average closeness, with less variation among top nodes
- Betweenness shows the most variation, identifying nodes that are critical connectors between different parts of the network

Figure 2: PageRank, Authority, and Hub Scores



Observations:

- **Hub and Authority scores are identical** because the graph is undirected. In undirected graphs, the HITS algorithm converges to the same values for both scores
- **PageRank shows higher variation** than Hub/Authority scores, making it more effective at discriminating between nodes in this graph type
- All three measures follow similar trends, but PageRank provides better granularity for ranking nodes
- The convergence of Hub and Authority scores confirms the symmetric nature of the Watts-Strogatz model

2.6 Key Insights

1. **Algorithm Selection Matters:** The choice between Girvan-Newman and Clauset-Newman-Moore has dramatic implications for computational feasibility at scale. While both detect communities, only CNM is practical for large networks.
2. **Scalability vs Accuracy Trade-off:** While Girvan-Newman may provide theoretically better community detection in some cases, its poor scalability ($O(m^2n)$) makes it impractical for real-world large networks beyond a few thousands nodes.
3. **Small-World Properties:** The Watts-Strogatz model successfully creates small-world networks with:
 - High clustering coefficient
 - Short average path lengths
 - Relatively uniform closeness centrality
 - Realistic structure similar to many real-world social networks
4. **Centrality Measure Selection:** Different centrality measures capture different aspects of node importance:
 - **PageRank:** Overall importance based on network structure
 - **Betweenness:** Bridge/connector role
 - **Closeness:** Accessibility to other nodes
 - **Hub/Authority:** Less useful for undirected graphs

Conclusion

This project successfully demonstrated:

1. **Graph Traversal Properties:** Implemented and tested functions to detect Euler paths and circuits based on fundamental graph theory principles (degree sequences and connectivity).
2. **Community Detection Scalability:** Empirically showed that algorithm complexity matters critically in practice:
 - Girvan-Newman: Theoretically sound but computationally infeasible for large graphs.
 - Clauset-Newman-Moore: Practical and scalable for real-world networks.
3. **Network Analysis:** Examined multiple centrality measures and their relationships, revealing the structural properties of small-world networks.

Practical Takeaway: For network analysis in production systems, scalability should be a primary consideration in algorithm selection. The "best" algorithm theoretically may not be the best choice practically. Real-world constraints (time, memory, computational resources) often dictate choosing faster approximation algorithms over theoretically optimal but slower exact algorithms.

Implementation Success: Using SNAP.py (Stanford Network Analysis Platform), we successfully implemented both problems according to assignment specifications, demonstrating proficiency with professional network analysis tools.